Dictionaries with Lists as Values

ISTA 130

Dictionaries

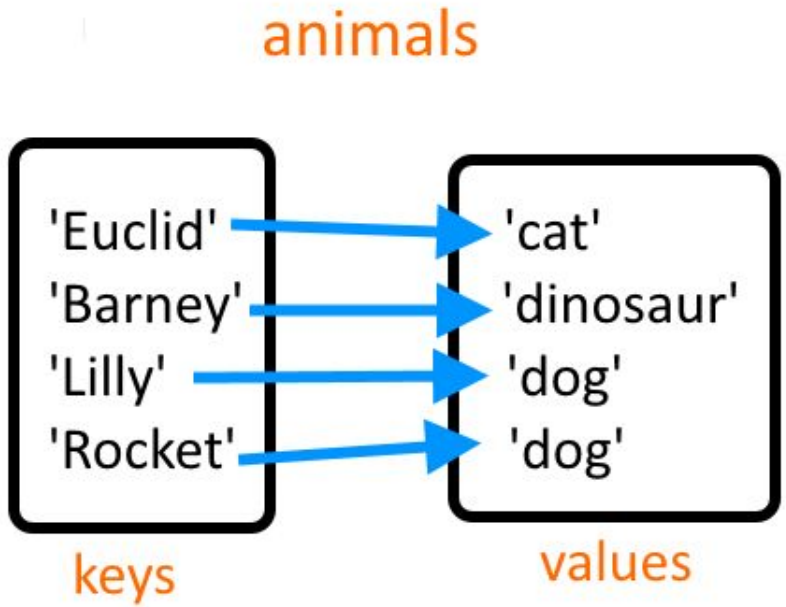
Like a list, a dictionary allows us to store multiple values in a single container.

- With a list we access those values via index numbers
  - you can think of an index as the location of an element within a list
- With a dictionary we access the values via `keys`
  - you can think of a key as the location of an element within a dictionary

Dictionaries

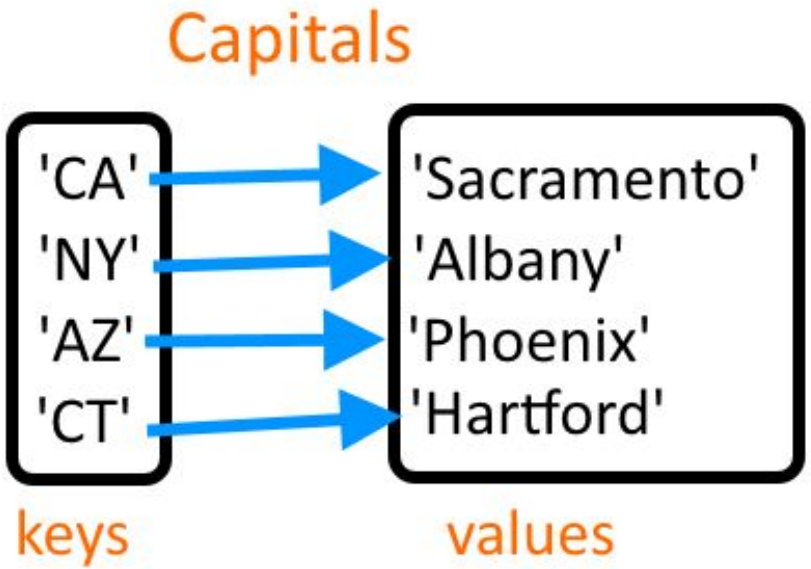
- Collection (set) of key-value pairs.
- Keys are unique.
- Create an empty dictionary: a_dict = {}
- Create a non-empty dictionary: a_dict = {'dog': 'Rocket', 'cat': 'Hobbes'}
- Add a key-value pair or change the value associated with a key: a_dict[key] = value
- Keys must be immutable; values can be anything.
- Dictionaries are mutable.
- Indexing on a non-existent key is an error, unless you're assigning to it.

Visualizing Dictionaries



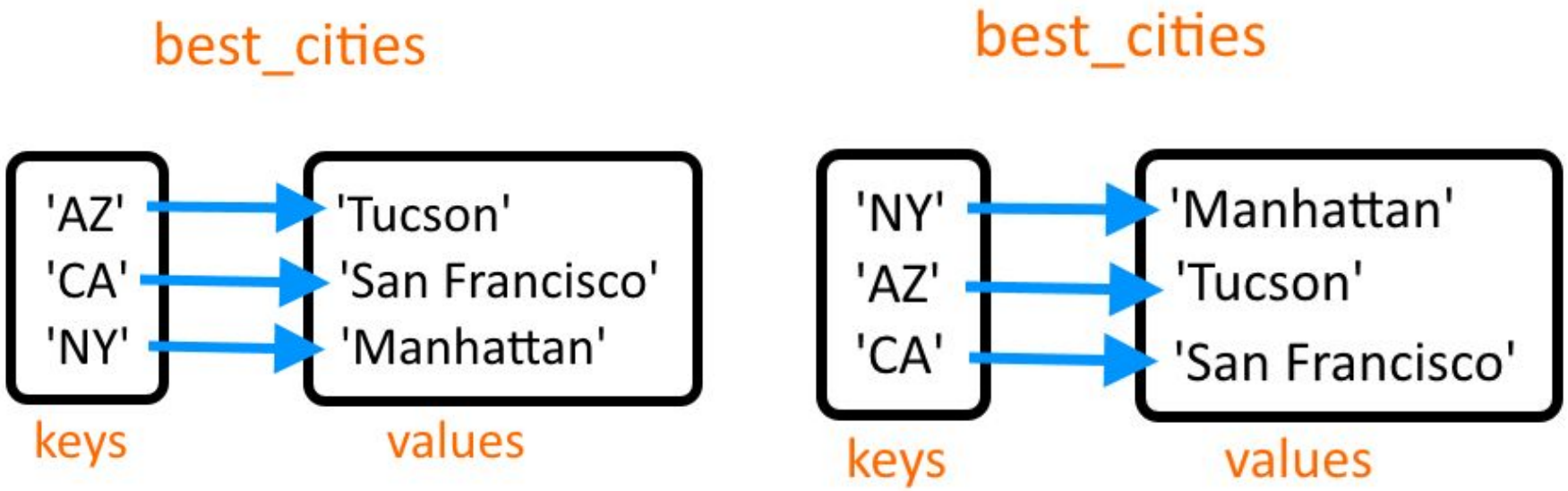

```
animals = {}
animals['Rocket'] = 'dog'
animals['Euclid'] = 'cat'
animals['Lilly'] = 'dog'
animals['Barney'] = 'dinosaur'
```

Drawing to Code




```
capitals = {}
capitals['CA'] = 'Sacramento'
capitals['NY'] = 'Albany'
capitals['AZ'] = 'Phoenix'
capitals['CT'] = 'Hartford'
```

These are the same




```
best_cities = {'AZ':'Tucson', 'CA':'San Francisco',
               'NY','Manhattan'}
```

Values can be any Type

```
stuff = {}
stuff['one'] = 1
stuff[42] = 'answer'
stuff['pi'] = 3.14159
stuff['fruit'] = ['kiwi', 'peach']
stuff['word_counts'] = {'the': 11, 'keep': 4}
```

Values can be any Type

```
stuff = {}
stuff['one'] = 1
stuff[42] = 'answer'
stuff['pi'] = 3.14159
stuff['fruit'] = ['kiwi', 'peach']
stuff['word_counts'] = {'the':  11, 'keep':  4}

print(stuff['fruit']) → ['kiwi', 'peach']

print(stuff['fruit'][1]) → 'peach'
```

Values can be any Type

```
stuff = {}
stuff['one'] = 1
stuff[42] = 'answer'
stuff['pi'] = 3.14159
stuff['fruit'] = ['kiwi', 'peach']
stuff['word_counts'] = {'the': 11, 'keep': 4}
```

```
print(stuff['word_counts']) →
   {'the':  11, 'keep':  4}

print(stuff['word_counts']['the']) → 11
```

Values can be any Type

```
stuff = {}
stuff['one'] = 1
stuff[42] = 'answer'
stuff['pi'] = 3.14159
stuff['fruit'] = ['kiwi', 'peach']
stuff['word_counts'] = {'the':  11, 'keep':  4}
```

```
print(stuff) →
{42:  'answer',
 'pi':  3.14159,
 'one':  1,
 'word_counts':  {'the':  11, 'keep':  4},
 'fruit':  ['kiwi', 'peach']}
```

Operator: `in`

- The **in** operator looks for **keys**, *not* values:

```
cheeses = {}
cheeses['Manchego'] = 'Spain'
cheeses['Leyden'] = 'Netherlands'
cheeses['Cabrales'] = 'Spain'
```

- So we can verify if a given *key* is in a dictionary:

```
print('Cabrales' in cheeses) → True
print('Spain' in cheeses) → False
print('Easy Cheese' in cheeses) → False
print('Easy Cheese' not in cheeses) → True
```

Example: Gradebook

Imagine we need to write a program to keep track of student grades

- Assume we can have any number of students
- Assume each student can have any number of grades
- How would store the data in our program?
  - list of students?
  - list of grades for each student?
  - how about a dictionary mapping each student name to a list of that student's grades?

Example: Lists as Values

Here is an excerpt of what some of the code might look like:

```
students = {}
...
name = input('Enter name:  ')
if name not in students:
   students[name] = []
grade = int(input('Enter grade:  '))
students[name].append(grade)
...
```

- If the `name` isn't a *key* in the gradebook dictionary yet:
  - add the name as a *key* with a new empty list as its *value*
  - we only do this *once* per student since each student only needs a single list
- Use the `name` to find the appropriate list to append a student grade to.

Example: Lists as Values

Here is code snippet showing one thing we might do with the `students` dictionary:

```
for name in students:
    grade_sum = sum(students[name])
    grade_count = len(students[name])
    mean = grade_sum / grade_count
    print(name + ':', mean)
```

Pet Dictionary




```
# initialize a pet list for key 'dog'
my_pets['dog'] = []
# add a dog's name
my_pets['dog'].append('Rocket')
```